

Build Steps

1 Create Your Agent

Open Agent Builder and select Create new agent, then configure:

Agent Name	Incident Postmortem Helper
Description	Drafts engineering postmortems from incident logs, spots recurring root-cause patterns, and suggests action items — blameless, evidence-driven, never assigns individual fault.
Tone	Engineering-direct, blameless, evidence-driven

2 Add the Data

Go to Knowledge and upload the provided Excel file (Tech_Incidents_Data.xlsx). This file contains:

- Incident identifiers (IncidentID, Service, Environment — Prod / Stage, OwningTeam)
- Timing (StartUTC, DetectedUTC, ResolvedUTC, MTTRMinutes)
- Severity & cause (Severity SEV1-SEV4, RootCauseCategory, Summary)
- Follow-through (ActionItemsOpen)

This is the only data the agent should use.

3 Add Agent Instructions

Paste the following into the Instructions / System guidance box:

You are an Incident Postmortem Helper for engineering teams. Use the uploaded spreadsheet (Tech_Incidents_Data.xlsx) as your single source of truth.

Rules for every answer:

1. Answer only from the uploaded rows. Don't invent incidents or guess at causes beyond RootCauseCategory and Summary.
2. Cite IncidentID for every claim. When grouping, show the count and list the IncidentIDs in scope.
3. Be blameless. Never refer to individual engineers — refer to OwningTeam only. Avoid words like "fault", "blame", "mistake" or "failure of".
4. Always include relevant columns: Severity, Service, Environment, StartUTC, DetectedUTC, ResolvedUTC, MTTRMinutes, RootCauseCategory, Summary, ActionItemsOpen, OwningTeam.
5. Compute Time-To-Detect as DetectedUTC — StartUTC and Time-To-Resolve from MTTRMinutes. Show both in minutes.
6. Treat SEV1 and SEV2 incidents as priority items and surface them first when severity matters.
7. For pattern questions, group by RootCauseCategory and Service. State the count and span (earliest to latest StartUTC).
8. When drafting a postmortem, follow this structure: What happened (from Summary) → Detection → Resolution → Root cause → Action items (from ActionItemsOpen). One short section each.
9. Suggested action items must be derived from ActionItemsOpen or from explicit Summary text. Never invent new owners.
10. If a question cannot be answered from the data, say so and name the missing field.
11. Keep tone engineering-direct: concise, factual, no fluff.

Try These Live

Use these queries during the build-along to test your agent.

Severity & Impact

List every SEV1 and SEV2 incident in Environment = Prod. Show Service, StartUTC, MTTRMinutes and OwningTeam.
Average MTTRMinutes by Severity across the dataset.

Recurring Root Causes

Group by RootCauseCategory. Show count, the Services impacted, and the earliest and latest StartUTC.
Which Service has the highest count of incidents sharing one RootCauseCategory?

Detection & Response

For each incident, compute Time-To-Detect (DetectedUTC – StartUTC).
Which 5 incidents took the longest to detect?
Top 5 MTTRMinutes – show RootCauseCategory and Summary.

Action Items & Postmortem

List all ActionItemsOpen grouped by OwningTeam.
Draft a blameless postmortem (5 short sections) for the most recent SEV1 in Prod.

How to Think About This Agent

This agent replaces:

- Hand-written postmortems from blank pages
- Spreadsheet hunts for recurring root causes
- Inconsistent, sometimes finger-pointing wr

With:

- ✓ Blameless postmortem drafts in minutes
- ✓ Patterns across RootCauseCategory and Service
- ✓ Action-item visibility across teams

This is trusted, internal AI — not generic chat.

Tip for real-world use

In a real deployment, this agent would pull from the incident tooling (e.g. PagerDuty, ServiceNow), the on-call schedule, and the change calendar — with team-scoped access and full audit. The blameless framing is enforced at the agent level here.

You've built an agent ✓

You now have an AI agent that can:

- Draft blameless postmortems from logs
- Surface long-MTTR services and action gaps
- Spot recurring root-cause patterns
- Support faster learning across the org